

VYAPAR AI

Intelligent WhatsApp-Style Commerce System with Local AI Automation

Complete Project & Implementation Report

Multi-tenant FastAPI modular monolith • Provider-agnostic AI layer • Local inference via Ollama • Multilingual
(English / Telugu / Hindi)

Generated 2026-08-14 • Status: business core + AI layer implemented; quality gates passing (434 tests, 90% coverage)

1. Executive Summary

Vyapar AI is a multi-tenant backend that lets a small business (a *dukaan*) run its daily operations — catalog, inventory, customers, orders, payments, invoices — and layers AI assistance on top: generating structured product drafts from an image, and answering natural-language merchant messages that are executed by validated backend services.

This report documents the whole system and, in particular, the completed AI layer: provider-agnostic **local inference through Ollama**, a **multilingual** conversational experience (English, Telugu, Hindi — native script, romanized, and code-mixed), a strict structured-output contract, a confidence/safety gate, and a measured evaluation suite.

The governing principle: AI understands language and proposes structured data; deterministic domain services remain the sole authority for executing business operations. The AI never writes to the database, never chooses the tenant, and never invents a business value.

Aspect	Summary
Stack	Python 3.12, FastAPI, PostgreSQL + SQLAlchemy + Alembic, Redis (auth only), Docker
Architecture	Modular monolith; router - schema - service - repository - model per domain
AI providers	mock gemini ollama (selected explicitly; no silent fallback)
Local models	qwen2.5:7b (chat), qwen2.5vl:3b (vision), mxbai-embed-large (embeddings)
Languages	English, Telugu, Hindi (native, romanized, code-mixed)
Tests	434 passing (+62 new), 90% coverage; ruff / mypy --strict / bandit clean

2. Problem & Solution

2.1 Problem

Small merchants juggle catalog, stock, orders, and billing across paper, spreadsheets, and chat. Inventory drifts, invoices are inconsistent, onboarding a catalog is slow, and routine questions (“do we have this?”, “add 10 units”) are repetitive manual work. Many such merchants are more comfortable typing in Telugu or Hindi than in English.

2.2 Solution

- **Deterministic business core** — every state change goes through validated domain services and database transactions.
- **AI assistance layer** — proposes structured data (catalog drafts, resolved intents) but never writes business data directly; a human approves drafts and intents execute only via the domain services after a confidence check.
- **Provider abstractions** — AI, payments, messaging, and storage sit behind interfaces, so the domain depends on protocols, not vendor SDKs.
- **Multilingual, local AI** — runs entirely on a local Ollama server; understands and replies in the merchant's language.

3. Technology Stack

Layer	Technology	Notes
Web / API	FastAPI + Uvicorn	Thin routers; OpenAPI docs; no frontend

Layer	Technology	Notes
Validation	Pydantic v2 / pydantic-settings	Strict schemas at every boundary
Persistence	PostgreSQL 16 + SQLAlchemy 2	Row-level locking for stock/payments
Migrations	Alembic	Single linear head; verified on clean DB
Auth store	Redis 7	Refresh-token revocation only (not caching)
AI (local)	Ollama 0.17.7 over HTTP	Structured JSON output, temperature 0
AI (hosted)	Gemini adapter	Optional; mock-tested only
Object storage	In-memory / S3 (MinIO)	Product images, invoice PDFs
PDF	ReportLab	Invoice rendering (and this report)
Tooling	ruff, mypy --strict, bandit, pytest	All gates enforced

4. System Architecture

4.1 Per-domain layering

```
router (thin HTTP) -> schemas (Pydantic) -> service (logic + transactions)
                    -> repository (tenant-scoped queries) -> models (SQLAlchemy)
```

All external infrastructure sits behind typing.Protocol abstractions, constructed in a single composition root (app/providers.py).

4.2 The AI request pipeline (the core boundary)

```
User message (any language)
  |
  v
ConversationService --> AiProvider --> Ollama / Gemini / Mock adapter
  |                                     |
  |                                     | Structured Intent (strict JSON)
  |                                     |
  |                                     v
  |                                     |
  |                                     | Pydantic re-validation
  |                                     | (confidence gate; tenant from JWT; ids/business_id stripped)
  |                                     |
  |                                     v
  |                                     |
  |                                     | Domain Service (authority): Catalog . Inventory . Order . Payment . Invoice
  |                                     |
  |                                     v
  |                                     |
  |                                     | Deterministic reply, in the user's language
```

Everything below the “Validation” line is deterministic and unit-tested. The AI influences only *which* operation and *which* language — never the values.

5. Architecture Principles & Invariants

- **Modular domain architecture** — one self-contained package per business domain.
- **Service / repository separation** — services own logic and transactions; repositories own tenant-scoped queries.
- **Tenant isolation** — every business-owned query filters by a `business_id` derived from the authenticated principal, never from client or AI input.
- **AI as assistance, not authority** — the model produces structured data only; domain services perform all writes.
- **Database-enforced invariants** — uniqueness, tenant scoping, and financial constraints at the schema level.
- **Transactional consistency & concurrency** — multi-step operations in one transaction; stock/payment updates take `SELECT . . . FOR UPDATE` row locks.
- **Explicit provider selection** — no silent fallback to a mock in production.

6. Domain Modules (Business Core)

Ten modules form the deterministic business core. They existed before the AI work and are behaviourally unchanged by it.

Module	Responsibility
auth + business	JWT, refresh rotation/revocation, RBAC (OWNER/EMPLOYEE/ADMIN), Business PIN, tenants
catalog	Categories, products, images, keyword/category filtering, soft delete
catalogai	Image -> AI draft -> merchant review/edit -> explicit approval -> product
inventory	Per-product stock, immutable movement log, row-locked single-writer, low-stock events
conversation	NL intent pipeline: search / stock / adjust (extended to multilingual here)
customer + order	Customers/addresses; order state machine with atomic inventory sync
payment	Online/COD payments, mock + Razorpay adapters, DB-enforced idempotency
invoice	Immutable numbered PDF invoice generated from a paid order
notification	In-app notifications persisted from domain events
analytics + dashboard	Read-only SQL aggregates and a composed KPI view

7. AI Provider Abstractions

There are two independent AI seams, each behind its own Protocol. The domain never imports a vendor SDK.

Protocol	Method	Implementations
AiProvider (catalog)	<code>describe(image, content_type) -> AiDraftPayload</code>	MockAiProvider, GeminiAdapter, OllamaAdapter
ConversationAiProvider	<code>resolve(text) -> ResolvedIntent</code>	MockConversationAiProvider, GeminiConversationAdapter, OllamaConversationAdapter
EmbeddingProvider	<code>embed(text) -> list[float]</code>	OllamaEmbeddingClient (scaffolding; not wired)

Each adapter maps failures to a typed exception hierarchy (*Timeout / *Unavailable / *RateLimited / *ConfigError / *InvalidResponse) so the service converts them into controlled, localized replies — never a stack trace, SQL, or provider internals to the user.

8. Ollama Integration (Local Inference)

Ollama is a concrete implementation of the existing abstractions, wired **only** in the composition root. Selecting it is a configuration choice, not a code change; no domain module imports Ollama.

8.1 Wire behaviour

- HTTP POST to {OLLAMA_BASE_URL}/api/generate with Ollama's structured format (a JSON schema) and temperature = 0 for deterministic, JSON-only output.
- The response is **re-validated** through ResolvedIntent / AiDraftPayload; malformed or hallucinated shapes become a typed error, never an unsafe action.
- The vision adapter sends the image as base64 in images[]; price is never requested and is dropped if emitted.

8.2 Configuration (environment-driven; no hard-coded model names)

```
AI_PROVIDER=ollama
OLLAMA_BASE_URL=http://localhost:11434
OLLAMA_CONVERSATION_MODEL=qwen2.5:7b
OLLAMA_CATALOG_MODEL=qwen2.5vl:3b
OLLAMA_EMBEDDING_MODEL=mxbai-embed-large
OLLAMA_REQUEST_TIMEOUT_SECONDS=60.0
```

Ollama can run on the host (recommended, uses the GPU directly) or as an optional docker-compose service behind the ollama profile — it is never baked into the application image.

9. Model Selection (Hardware-Aware)

Detected development hardware: **16 CPU cores, 15.3 GB RAM, NVIDIA RTX 3050 Laptop with 6 GB VRAM**. The 6 GB VRAM ceiling is the binding constraint, so a Q4 7B model (~4.7 GB) is the largest that fits comfortably with context.

Role / Setting	Model	Size	Why
Conversation (default)	qwen2.5:7b	4.7 GB	Strong multilingual (Hindi/Telugu) + strict JSON; fits 6 GB VRAM
Conversation (light)	qwen2.5:3b-instruct / llama3.2:3b	1.9 / 2.0 GB	Lower latency where some accuracy can be traded
Catalog vision	qwen2.5vl:3b	3.2 GB	Compact VLM for drafting name/desc/tags
Embeddings	mxbai-embed-large	0.67 GB	Scaffolding; bge-m3 recommended for prod

Fallback: if Ollama is unreachable or a model is not pulled, the adapter raises a typed error and the service replies with a controlled “please try again” message in the user’s language — it never fabricates a result. AI_PROVIDER=mock is rejected in production.

10. Multilingual Support

The assistant understands English, Telugu, and Hindi — in native script, romanized (Latin), and code-mixed — and maps all of them to the same language-neutral structured intent. Language is metadata: it never changes the operation or the numbers.

10.1 Same intent, three languages (romanized shown; native script works in the app)

Message	Intent	Entities
add 20 notebooks (EN)	ADJUST_STOCK	qty 20, INCREASE
20 notebooks add cheyyi (TE, romanized)	ADJUST_STOCK	qty 20, INCREASE
20 notebook add karo (HI, romanized)	ADJUST_STOCK	qty 20, INCREASE
notebook kitne hain? (HI)	GET_STOCK	product notebook

10.2 Who decides the reply language

The reply language is derived **deterministically from the user’s own text** (app/conversation/language.py), not from the model, so it cannot be spoofed: Telugu Unicode block -> Telugu; Devanagari -> Hindi; Latin/undetermined -> the model’s language hint if given, else English. Measured language detection: **100%**.

10.3 Responses never invent business values

Replies are built from per-language template tables into which the domain service’s real values are interpolated (app/conversation/responses.py). The AI chooses the template and language; it never supplies the stock number, price, or status. English output is byte-identical to the original single-language wording, preserving existing behaviour.

11. Structured Output & Safety

- **Strict schema.** The model must return `ResolvedIntent`: intent, confidence (0..1), entities (product_query, quantity, direction, movement_type), language, optional clarification. `extra="ignore"` drops any fabricated business_id, product_id, SQL, or tool call before it can reach a handler.
- **Tenant safety.** The intent carries no tenant/id; business_id and actor_user_id come only from the JWT principal.
- **Confidence gate.** Low-confidence actionable intents never mutate — they ask for clarification, in any language.
- **Prompt-injection containment.** “ignore instructions”, “delete all stock”, “switch business” (in any language) are classified UNSUPPORTED; even a misclassification is contained because a mutation still requires a resolved product, a positive quantity, and a passing confidence check.
- **Sanitized errors.** Provider/domain failures become generic replies; no internals leak.

12. Evaluation

An evaluation suite (evals/) separates **model accuracy** (does the provider return the right structured intent?) from **business-operation correctness** (does the domain service return the right number?), which is proven by integration tests. The 31-case dataset spans English, Telugu, and Hindi in native, romanized, code-mixed, misspelled, colloquial, incomplete, ambiguous, unsupported, and prompt-injection forms.

Run it with: `python -m evals.run_eval --provider ollama`. Results below are from **real local inference on qwen2.5:7b** (not estimates).

Metric	Result
Intent accuracy	93.5%
Quantity extraction	100%
Direction extraction	100%
Product resolution	95.2%
Model language detection	100%
Unsupported-request detection	100%
Prompt-injection rejection (model layer)	100%
Provider errors	0 / 31
Average latency per request	~6.3 s (RTX 3050, 6 GB VRAM)
Structured-output success	31 / 31 (0 malformed)

13. Security & Multi-Tenancy

- **Authentication** — JWT access tokens; refresh-token rotation with revocation (Redis in staging/production).
- **Authorization** — role checks (OWNER/EMPLOYEE/ADMIN); the conversation endpoint requires OWNER/EMPLOYEE (it can mutate stock); ADMIN is denied.
- **Tenant scoping** — every query is filtered by the principal's business; cross-tenant access returns not-found. Verified by a Telugu cross-tenant test in the suite.
- **Secrets** — environment-sourced; passwords/PINs bcrypt-hashed; none in code.
- **Payments** — currency and business identity are server-trusted, not client-supplied; state transitions are row-locked and idempotent.

14. Testing & Quality Gates (Actually Executed)

Gate	Result
ruff check	Pass
ruff format --check	Pass (198 files)
mypy --strict (app + evals)	Pass (121 + 4 files)
bandit	Pass (0 issues)
pytest (full suite)	434 passed (was 372; +62 new)
Coverage	90%
Docker build	Pass
Docker Compose config (incl. ollama profile)	Valid

Gate	Result
Live Ollama connectivity + model eval	Pass (real inference)
Alembic	Unchanged (no DB schema change; language is Pydantic-only)

New tests cover: Ollama adapter parsing & failure mapping (success, timeout, unavailable, malformed JSON, invalid schema, rate-limit, model-not-found), the vision adapter, deterministic language detection, multilingual response templates, the embedding client, dataset integrity, and an end-to-end multilingual integration test (Telugu/Hindi drive real inventory mutations and localized replies, with cross-tenant isolation).

15. Honest Status — Implemented vs. Mocked vs. Not Done

Capability	Status
Business core (10 modules)	Implemented
Conversational AI (multilingual EN/TE/HI)	Implemented; measured LIVE on Ollama qwen2.5:7b
Catalog AI vision (Ollama)	Implemented + unit-tested (mock HTTP); not benchmarked live
Embedding client / semantic search	Scaffolding only; not wired (no vector DB by design)
Gemini adapters	Implemented; mock-tested only (no live credentials run)
Razorpay payment adapter	Implemented; mock-tested only (no live gateway run)
WhatsApp / Meta messaging	NOT implemented (mock provider only; selecting it raises)
External notification delivery (WhatsApp/email)	NOT implemented (in-app only)
Durable event delivery	NOT implemented (in-process EventBus; no outbox/broker)

“WhatsApp-based” describes the intended channel and message shape, not a shipped Meta integration. No claim is made that WhatsApp, Gemini, or Razorpay were verified live.

16. Recommended Next Steps

- Implement a real **WhatsApp Cloud API** adapter for the MessagingProvider (webhook verify, inbound parse, outbound send) — the single biggest gap for the WhatsApp-commerce vision.
- Live-verify **Gemini** and **Razorpay** with real credentials in a staging secret store.
- Benchmark the **Ollama vision** catalog path on real product photos.
- Wire **multilingual semantic search** with a tenant-scoped pgvector column; upgrade embeddings to bge-m3.
- Offer the 3B conversation model by default on low-VRAM hosts; consider response streaming.
- Add a **transactional outbox** if notification delivery must be guaranteed.

17. How to Run

```
# 1. Infrastructure
docker compose up -d db redis minio

# 2. Application (host)
pip install -e ".[dev,storage,pdf]"
alembic upgrade head
uvicorn app.main:app --reload

# 3. Local AI (host Ollama recommended)
ollama pull qwen2.5:7b
# .env: AI_PROVIDER=ollama, OLLAMA_BASE_URL=http://localhost:11434

# 4. Verify AI quality (writes docs/ai/evaluation_report.md)
python -m evals.run_eval --provider ollama
```

18. File Inventory (This AI Work)

Created

app/conversation/adapters/ollama.py, app/catalogai/adapters/ollama.py, app/conversation/embeddings.py, app/conversation/language.py; evals/ (dataset, harness, run_eval); tests/unit/test_{conversation_ollama, catalogai_ollama, language, responses_multilingual, ollama_embeddings, eval_dataset}.py; tests/integration/test_conversation_multilingual.py; docs/PROJECT_OVERVIEW.md; docs/ai/{architecture_assessment, ollama, model_selection, multilingual, evaluation_report}.md; this report.

Modified

app/config.py, app/providers.py, app/conversation/{schemas, responses, handlers, service}.py, docker-compose.yml, .env.example, README.md. No database schema change; Alembic untouched.

Vyapar AI — the AI is an intelligence layer; the domain services remain the authority. AI assists users by understanding natural-language requests and generating structured information, while validated backend services execute business operations.